

Section 8.1: Domain-Driven Design (DDD)

Module 8 of 8 · Strategic & Tactical Design Patterns

Executive Overview

Domain-Driven Design (DDD) aligns software architecture with business capabilities. It provides a language and set of patterns for decomposing complex systems into manageable, coherent units. Mastering bounded contexts, aggregates, and context integration patterns is essential for senior-level system design interviews.

Part 1: Strategic Design

1.1 Bounded Contexts

A bounded context is an explicit boundary within which a particular domain model applies. Outside that boundary, the same word may mean something entirely different — or nothing at all.

Canonical E-Commerce Example:

Context	Owns	“Product” Means
Catalog	Description, photos, SEO	Displayable item with rich content
Inventory	Stock levels, SKUs	A countable physical unit
Order	Line items, pricing	A priced line in a transaction
Shipping	Weight, dimensions	A physical package to be routed

Interview tip: When a candidate maps all contexts to a single “Product” entity, that is the signal that strategic design has not been applied. Each context owns its own model.

1.2 Context Mapping Patterns — When, Where & How

These patterns describe the *relationship* between two bounded contexts. Choosing the wrong pattern leads to tight coupling and fragility.

Pattern	When to Use	How It Works	Tradeoffs
Shared Kernel	Two teams own closely overlapping models	Share a library/module; changes require coordination	High coordination cost; avoid unless teams are co-located
Customer–Supplier	Upstream can plan roadmap around downstream needs	Upstream provides API with agreed SLA; downstream tests against it	Downstream depends on upstream schedule
Conformist	Downstream cannot influence upstream (e.g. legacy ERP)	Downstream adopts upstream model wholesale	Upstream model leaks into downstream; technical debt risk
Anti-Corruption Layer (ACL)	Integrating a legacy or external system with an incompatible model	Facade or adapter translates between models	Additional layer, but protects domain purity
Open Host Service	Many consumers need the same upstream context	Publish a stable, versioned API and shared language	Requires discipline to maintain; versioning overhead
Published Language	Combined with Open Host; formal interchange format needed	Use a standard schema (JSON-LD, Protobuf, Avro)	Schema evolution requires coordination

Implementation — Anti-Corruption Layer:

```
// Protects Order context from legacy ERP model
class LegacyErpOrderAdapter implements OrderRepository {
    private LegacyErpClient erpClient;

    public Order find(OrderId id) {
        ErpSalesDoc doc = erpClient.getSalesDocument(id.value());
        // Translate ERP concept → clean domain model
        return new Order(
            OrderId.of(doc.getDocNumber()),
            Money.of(doc.getNetValue(), doc.getCurrency()),
            mapStatus(doc.getProcessingStatus())
        );
    }
}
```

```

private OrderStatus mapStatus(String erpStatus) {
    return switch (erpStatus) {
        case "A" -> OrderStatus.PENDING;
        case "B" -> OrderStatus.CONFIRMED;
        case "C" -> OrderStatus.SHIPPED;
        default -> OrderStatus.UNKNOWN;
    };
}
}

```

1.3 Ubiquitous Language

The ubiquitous language is a shared vocabulary between domain experts, engineers, and QA — used consistently in conversations, code, and tests.

Example — “Order” in three dialects:

Role	Phrase	Underlying Concept
Business	“Customer places an order”	Intentional purchase act
Engineer	Order aggregate created with status PENDING	Aggregate root instantiated
QA	POST /api/orders → 201 with order_id	API contract verified

All three refer to the same concept. The language unifies them.

1.4 Subdomain Classification

Type	Description	Investment Level	Example
Core	Primary differentiator; competitive advantage	High — custom build	Netflix recommendation engine
Supporting	Necessary but not differentiating	Medium — internal build or adapt	User authentication service
Generic	Commodity; solved problems	Low — buy or use OSS	Email delivery, file storage

Interview tip: Misidentifying a core subdomain as generic (and buying a SaaS for it) destroys competitive advantage. Misidentifying a generic subdomain as core wastes engineering capital.

Part 2: Tactical Design

2.1 Entities vs Value Objects

Entities have identity that persists over time and state changes. Two entities with the same data are *not* the same if their IDs differ.

Value Objects are defined entirely by their attributes. Two value objects with the same attributes *are* identical. They must be immutable.

```
// Entity – identity via UserId
class User {
    private final UserId id;
    private String name;
    private Email email;

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof User)) return false;
        return this.id.equals(((User) o).id); // Identity-based
    }
}

// Value Object – equality via content
class Money {
    private final BigDecimal amount;
    private final Currency currency;

    public Money add(Money other) {
        if (!this.currency.equals(other.currency))
            throw new CurrencyMismatchException();
        return new Money(this.amount.add(other.amount), this.currency); // New
object
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Money)) return false;
        Money m = (Money) o;
        return this.amount.compareTo(m.amount) == 0
            && this.currency.equals(m.currency); // Value-based
    }
}
```

```
}  
}
```

2.2 Aggregates

An aggregate is a cluster of entities and value objects with a single **aggregate root** that controls all mutations and enforces invariants. External objects hold references only to the root.

Order Aggregate:

```
class Order { // Aggregate root  
    private final OrderId id;  
    private final List<OrderLine> lines = new ArrayList<>();  
    private Money total = Money.ZERO;  
    private static final Money MAX_ORDER = Money.of(10_000, Currency.USD);  
  
    // Only root controls mutations  
    public void addLine(Product product, int quantity) {  
        Money lineTotal = product.price().multiply(quantity);  
        Money newTotal = total.add(lineTotal);  
  
        // Invariant: total must not exceed max  
        if (newTotal.isGreaterThan(MAX_ORDER))  
            throw new OrderLimitExceededException(newTotal, MAX_ORDER);  
  
        lines.add(new OrderLine(product.id(), quantity, lineTotal));  
        total = newTotal;  
    }  
  
    public void removeLine(ProductId productId) {  
        OrderLine line = lines.stream()  
            .filter(l -> l.productId().equals(productId))  
            .findFirst()  
            .orElseThrow(LineNotFoundException::new);  
        lines.remove(line);  
        total = total.subtract(line.lineTotal());  
    }  
}
```

Key rule: No external service reaches inside an aggregate to modify `OrderLine` directly. All changes flow through `Order`.

2.3 Domain Events

Domain events represent something meaningful that *happened* in the domain — past tense, immutable facts.

```
// Immutable record of what happened
public record OrderPlaced(
    OrderId orderId,
    CustomerId customerId,
    Money total,
    List<OrderLine> lines,
    Instant occurredAt
) implements DomainEvent {}

// Published after successful aggregate state change
class Order {
    private final List<DomainEvent> events = new ArrayList<>();

    public void place() {
        if (lines.isEmpty()) throw new EmptyOrderException();
        this.status = OrderStatus.PLACED;
        events.add(new OrderPlaced(id, customerId, total, List.copyOf(lines),
Instant.now()));
    }

    public List<DomainEvent> pullEvents() {
        var copy = List.copyOf(events);
        events.clear();
        return copy;
    }
}
}
```

2.4 Repositories

Repositories provide a collection-like interface for retrieving and persisting aggregates. They hide all storage concerns from the domain.

```
// Domain defines the contract
interface OrderRepository {
    Order findById(OrderId id);
    void save(Order order);
    List<Order> findByCustomer(CustomerId customerId);
    List<Order> findPendingOlderThan(Duration age);
}

// Infrastructure provides the implementation
class JpaOrderRepository implements OrderRepository {
    private final EntityManager em;

    @Override
    public Order findById(OrderId id) {
        OrderEntity entity = em.find(OrderEntity.class, id.value());
        if (entity == null) throw new OrderNotFoundException(id);
        return OrderMapper.toDomain(entity); // Map persistence → domain
    }
}
```

```

@Override
public void save(Order order) {
    OrderEntity entity = OrderMapper.toEntity(order);
    em.merge(entity);
    // Publish domain events after commit
    order.pullEvents().forEach(eventBus::publish);
}
}

```

Part 3: DDD & Microservice Decomposition

3.1 Bounded Context → Service Boundary

Each bounded context is a *candidate* for a microservice, not an automatic one. Consider:

Factor	Favour Extraction	Favour Keeping Together
Team structure	Different teams own contexts	Single team owns both
Change frequency	Different deployment cycles	Deploy together always
Data consistency	Eventual consistency acceptable	Strong ACID required
Traffic patterns	Very different load profiles	Similar load profiles

3.2 Strangler Fig Pattern

Migrating a monolith with DDD boundaries:

```

Phase 1: Identify context boundaries in the monolith
        Monolith: [Order][Inventory][Payment][Shipping]

Phase 2: Extract one context at a time
        Service: [Order Service]
        Monolith: [Inventory][Payment][Shipping]
        API Gateway: routes /orders → Order Service, rest → Monolith

Phase 3: Repeat until monolith is gone
        All contexts extracted; monolith deleted

```

Part 4: Interview Design Problems

Problem 1 — Multi-Context E-Commerce System

Question: Design a multi-context e-commerce system using DDD. What are the key boundaries, integration points, and consistency guarantees?

Solution:

Bounded Contexts:

1. **Catalog Context** — products, categories, search; read-only for all other contexts
2. **Order Context** — cart lifecycle, pricing, discounts; publishes `OrderPlaced`
3. **Inventory Context** — stock levels, reservations; consumes `OrderPlaced`, publishes `StockReserved` / `StockReservationFailed`
4. **Payment Context** — transactions, refunds; consumes `StockReserved`, publishes `PaymentCompleted` / `PaymentFailed`
5. **Shipping Context** — packages, carriers, tracking; consumes `PaymentCompleted`

Integration via Domain Events (choreography):

```
// Inventory subscribes to Order events
@EventHandler
class InventoryEventHandler {
    private final InventoryService inventoryService;
    private final EventBus eventBus;

    void on(OrderPlaced event) {
        try {
            inventoryService.reserve(event.orderId(), event.lines());
            eventBus.publish(new StockReserved(event.orderId()));
        } catch (InsufficientStockException e) {
            eventBus.publish(new StockReservationFailed(event.orderId(),
e.reason()));
        }
    }
}

// ACL: Order ↔ Inventory translation
class OrderInventoryAcl {
    List<InventoryReservationItem> translate(List<OrderLine> lines) {
        return lines.stream()
            .map(l -> new InventoryReservationItem(l.productId().value(),
l.quantity()))
            .toList();
    }
}
```


Consistency model: Each context achieves ACID internally; cross-context consistency is eventual via the event chain. Compensating transactions (e.g., `StockReleased` on payment failure) handle rollback.

Problem 2 — Legacy System Integration

Question: Your new Order Service must integrate with a 10-year-old ERP that models “sales documents” instead of “orders.” How do you prevent the legacy model from corrupting your domain?

Solution:

Apply an **Anti-Corruption Layer** with explicit translation:

```
// The ERP speaks "SalesDocument"
interface ErpClient {
    ErpSalesDoc getSalesDocument(String docNumber);
    void updateSalesDocument(ErpSalesDoc doc);
}

// The ACL translates to/from clean domain model
@Component
class ErpOrderAdapter implements OrderRepository {
    private final ErpClient erp;

    @Override
    public Order findById(OrderId id) {
        ErpSalesDoc doc = erp.getSalesDocument(id.value());

        // Translation: ERP → Domain
        List<OrderLine> lines = doc.getItems().stream()
            .map(item -> new OrderLine(
                ProductId.of(item.getMaterialNumber()),
                item.getOrderQuantity().intValue(),
                Money.of(item.getNetPrice(), doc.getCurrency())
            ))
            .toList();

        return new Order(id, CustomerId.of(doc.getSoldToParty()), lines);
    }

    @Override
    public void save(Order order) {
        // Translation: Domain → ERP
        ErpSalesDoc doc = new ErpSalesDoc();
        doc.setDocNumber(order.id().value());
        doc.setProcessingStatus(translateStatus(order.status()));
        // ...
        erp.updateSalesDocument(doc);
    }
}
```

```
}  
}
```

The domain model never imports `ErpSalesDoc`. All ERP concepts stop at the adapter boundary.

Problem 3 — Aggregate Boundary Design

Question: In an order system, should `Order` and `Customer` be in the same aggregate? What goes wrong if they are?

Solution:

They should **not** be in the same aggregate. The rule is: aggregate boundaries should match *transactional consistency requirements*, not just domain relationships.

```
Wrong aggregate:  
CustomerAggregate {  
    Customer root  
    List<Order> orders    // Could be thousands!  
    Address address  
}
```

Problem 1 — Concurrency: Every time ANY order changes, the entire Customer aggregate must lock.
Two concurrent order placements → conflict.

Problem 2 — Size: A customer with 10,000 orders loads 10,000 order records every time you update the address.

Problem 3 — Invariants: What invariant requires Customer and Order to change atomically? Almost none.

```
// Correct: separate aggregates, linked by ID only  
class Order {  
    private OrderId id;  
    private CustomerId customerId; // Reference by ID, not object  
    private List<OrderLine> lines;  
}  
  
class Customer {  
    private CustomerId id;  
    private String name;  
    private Address address;  
    // No reference to orders  
}  
  
// Use a domain service if cross-aggregate logic is needed  
class OrderPlacementService {
```

```

    public Order placeOrder(CustomerId customerId, List<CartItem> items) {
        Customer customer = customerRepo.findById(customerId);
        customer.validateCanOrder();    // Check credit limit etc.
        Order order = Order.create(customerId, items);
        orderRepo.save(order);
        return order;
    }
}

```

Rule of thumb: If you find yourself loading Aggregate A just to update Aggregate B, they should be separate aggregates connected by ID.

Quick Reference

Concept	One-Line Definition
Bounded Context	Explicit boundary within which a model applies
Ubiquitous Language	Shared vocabulary used identically in code and conversation
Entity	Object defined by identity; mutable over time
Value Object	Object defined by attributes; immutable
Aggregate	Consistency boundary with a single root
Domain Event	Immutable record of a meaningful past occurrence
Repository	Collection-like abstraction for aggregate persistence
ACL	Translator between incompatible domain models
Strangler Fig	Incremental migration pattern from monolith to services